TP 2 - Monolithe Modulaire
Restructuration et Préparation à la Migration
EL KHAYAR Youssef

Étape 1 : Ajouter MapStruct:

J'ai ajouté la dépendance MapStruct dans le pom.xml ainsi que le plugin maven-compiler-plugin pour permettre la génération automatique du code de mapping entre les entités et les DTOs.

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSch
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.2.5</version>
        <relativePath/>
    </parent>
    <groupId>com.ecommerce</groupId>
    <artifactId>monolith</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>monolith</name>
    <description>Projet E-commerce Monolithe</description>

    <properties>
        <java.version>17</java.version>
        <org.mapstruct.version>1.5.5.Final</org.mapstruct.version>
    </properties>
```

```
22      <dependencies> Add Starters...
25          <artifactId>spring-boot-starter-data-jpa</artifactId>
26      </dependency>
27      <dependency>
28          <groupId>org.postgresql</groupId>
29          <artifactId>postgresql</artifactId>
30          <scope>runtime</scope>
31      </dependency>
32      <dependency>
33          <groupId>org.springframework.boot</groupId>
34          <artifactId>spring-boot-starter-web</artifactId>
35      </dependency>
36      <dependency>
37          <groupId>org.springframework.boot</groupId>
38          <artifactId>spring-boot-starter-validation</artifactId>
39      </dependency>
40      <dependency>
41          <groupId>org.projectlombok</groupId>
42          <artifactId>lombok</artifactId>
43          <optional>true</optional>
44      </dependency>
45      <dependency>
46          <groupId>org.springframework.boot</groupId>
47          <artifactId>spring-boot-starter-test</artifactId>
48          <scope>test</scope>
47          <artifactId>spring-boot-starter-test</artifactId>
48          <scope>test</scope>
49      </dependency>
50      <dependency>
51          <groupId>org.springframework.boot</groupId>
52          <artifactId>spring-boot-starter-data-rest</artifactId>
53      </dependency>
54      <dependency>
55          <groupId>org.mapstruct</groupId>
56          <artifactId>mapstruct</artifactId>
57          <version>${org.mapstruct.version}</version>
58      </dependency>
59  </dependencies>
```

```
62            <plugins>
63                <plugin>
64                    <groupId>org.springframework.boot</groupId>
65                    <artifactId>spring-boot-maven-plugin</artifactId>
66                    <configuration>
67                        <excludes>
68                            <exclude>
69                                <groupId>org.projectlombok</groupId>
70                                <artifactId>lombok</artifactId>
71                            </exclude>
72                        </excludes>
73                    </configuration>
74                </plugin>
75                <plugin>
76                    <groupId>org.apache.maven.plugins</groupId>
77                    <artifactId>maven-compiler-plugin</artifactId>
78                    <version>3.11.0</version>
79                    <configuration>
80                        <source>17</source>
81                        <target>17</target>
82                        <annotationProcessorPaths>
83                            <path>
84                                <groupId>org.projectlombok</groupId>
85                                <artifactId>lombok</artifactId>
```

```
85                                <artifactId>lombok</artifactId>
86                                <version>1.18.30</version>
87                            </path>
88                            <path>
89                                <groupId>org.mapstruct</groupId>
90                                <artifactId>mapstruct-processor</artifactId>
91                                <version>1.5.5.Final</version>
92                            </path>
93                        </annotationProcessorPaths>
94                    </configuration>
95                </plugin>
96            </plugins>
97        </build>
98    </project>
```

Étape 2 : Créer les DTOs :

J'ai créé ProductDTO pour exposer les données en sortie et CreateProductRequest pour recevoir les données en entrée, séparant ainsi la couche de présentation de la couche métier.

ProductDTO.java:

```java
package com.ecommerce.monolith.product.dto;

import ...

@Data  10 usages
public class ProductDTO {
    private Long id;
    private String name;
    private String description;
    private BigDecimal price;
    private Integer stock;
}
```

ProductDTO.java:

```java
package com.ecommerce.monolith.product.dto;

import ...

@Data  6 usages
public class CreateProductRequest {
    @NotBlank(message = "Product name is required")
    private String name;
    private String description;
    @NotNull(message = "Price is required")
    @Positive(message = "Price must be greater than 0")
    private BigDecimal price;
    @PositiveOrZero(message = "Stock cannot be negative")
    private Integer stock = 0;
}
```

Étape 3 : Créer le Mapper:

J'ai créé ProductMapper qui assure la conversion entre les entités et les DTOs, ce qui évite d'exposer directement les entités JPA dans les réponses de l'API.

```java
package com.ecommerce.monolith.product.mapper;

import com.ecommerce.monolith.product.dto.CreateProductRequest;
import com.ecommerce.monolith.product.dto.ProductDTO;
import com.ecommerce.monolith.product.model.Product;
import org.springframework.stereotype.Component;
import java.util.List;
import java.util.stream.Collectors;

@Component  3 usages
public class ProductMapper {

    public ProductDTO toDTO(Product product) {  5 usages
        ProductDTO dto = new ProductDTO();
        dto.setId(product.getId());
        dto.setName(product.getName());
        dto.setDescription(product.getDescription());
        dto.setPrice(product.getPrice());
        dto.setStock(product.getStock());
        return dto;
    }

    public List<ProductDTO> toDTOList(List<Product> products) {  1 usage
        return products.stream()  Stream<Product>
                .map(this::toDTO)  Stream<ProductDTO>
                .collect(Collectors.toList());
    }

    public Product toEntity(CreateProductRequest request) {  1 usage
        Product product = new Product();
        product.setName(request.getName());
        product.setDescription(request.getDescription());
        product.setPrice(request.getPrice());
        product.setStock(request.getStock());
        return product;
    }

    public void updateEntity(CreateProductRequest request, Product product) {  1 usage
        product.setName(request.getName());
        product.setDescription(request.getDescription());
        product.setPrice(request.getPrice());
        product.setStock(request.getStock());
    }
}
```

Étape 4 : Refactorer le Service:

J'ai séparé le service en deux parties : une interface ProductService qui définit le contrat, et ProductServiceImpl qui contient l'implémentation réelle utilisant le mapper et le repository.

ProductService.java

```java
package com.ecommerce.monolith.product.service;

import com.ecommerce.monolith.product.dto.CreateProductRequest;
import com.ecommerce.monolith.product.dto.ProductDTO;
import java.util.List;

public interface ProductService {  3 usages  1 implementation
    List<ProductDTO> getAllProducts();  1 usage  1 implementation
    ProductDTO getProductById(Long id);  1 usage  1 implementation
    ProductDTO createProduct(CreateProductRequest request);  1 usage  1 implementation
    ProductDTO updateProduct(Long id, CreateProductRequest request);  1 usage  1 implementation
    void deleteProduct(Long id);  1 usage  1 implementation
}
```

ProductServiceImpl.java

```java
package com.ecommerce.monolith.product.service;

import com.ecommerce.monolith.product.dto.CreateProductRequest;
import com.ecommerce.monolith.product.dto.ProductDTO;
import com.ecommerce.monolith.product.mapper.ProductMapper;
import com.ecommerce.monolith.product.model.Product;
import com.ecommerce.monolith.product.repository.ProductRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
@RequiredArgsConstructor
public class ProductServiceImpl implements ProductService {
    private final ProductRepository repository;
    private final ProductMapper mapper;

    @Override  1 usage
    public List<ProductDTO> getAllProducts() {
        return mapper.toDTOList(repository.findAll());
    }
```

```java
    @Override  1 usage
    public ProductDTO getProductById(Long id) {
        Product product = repository.findById(id)
                .orElseThrow(() -> new RuntimeException("Product not found with id: " + id)
        return mapper.toDTO(product);
    }

    @Override  1 usage
    public ProductDTO createProduct(CreateProductRequest request) {
        Product product = mapper.toEntity(request);
        return mapper.toDTO(repository.save(product));
    }

    @Override  1 usage
    public ProductDTO updateProduct(Long id, CreateProductRequest request) {
        Product product = repository.findById(id)
                .orElseThrow(() -> new RuntimeException("Product not found with id: " + id)
        mapper.updateEntity(request, product);
        return mapper.toDTO(repository.save(product));
    }

    @Override  1 usage
    public void deleteProduct(Long id) {
        if (!repository.existsById(id))
            throw new RuntimeException("Product not found with id: " + id);
        repository.deleteById(id);
    }
}
```

Étape 5 : Adapter le Controller:

J'ai modifié le controller pour qu'il utilise désormais CreateProductRequest en entrée et ProductDTO en sortie au lieu des entités directement, ce qui respecte le principe de séparation des couches.

```java
package com.ecommerce.monolith.product.controller;

import com.ecommerce.monolith.product.dto.CreateProductRequest;
import com.ecommerce.monolith.product.dto.ProductDTO;
import com.ecommerce.monolith.product.service.ProductService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/products")
@RequiredArgsConstructor
public class ProductController {
    private final ProductService productService;

    @GetMapping
    public ResponseEntity<List<ProductDTO>> getAll() {
        return ResponseEntity.ok(productService.getAllProducts());
    }

    @GetMapping("/{id}")
    public ResponseEntity<ProductDTO> getById(@PathVariable Long id) {
        return ResponseEntity.ok(productService.getProductById(id));
    }

    @PostMapping
    public ResponseEntity<ProductDTO> create(@Valid @RequestBody CreateProductRequest reque
        return ResponseEntity.status(HttpStatus.CREATED).body(productService.createProduct(
    }

    @PutMapping("/{id}")
    public ResponseEntity<ProductDTO> update(@PathVariable Long id, @Valid @RequestBody Cre
        return ResponseEntity.ok(productService.updateProduct(id, request));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        productService.deleteProduct(id);
        return ResponseEntity.noContent().build();
    }
}
```

Tester avec Postman:

http://localhost:8080/api/products

Save Share

POST http://localhost:8080/api/products Send

Docs Params Authorization Headers 9 Body Scripts Tests Settings Cook

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "name": "Laptop",
3   "description": "Un bon laptop",
4   "price": 999.99,
5   "stock": 10
6 }
```

Body Cookies Headers 5 Test Results 201 Created 1.35 s 249 B

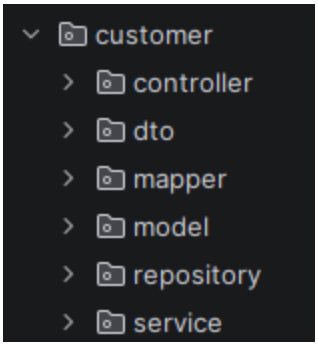
{} JSON Preview Visualize

```
1 {
2     "id": 8,
3     "name": "Laptop",
4     "description": "Un bon laptop",
5     "price": 999.99,
6     "stock": 10
7 }
```

Exercice:

Partie 1 — Module Customer:

J'ai créé le module Customer complet avec son entité, DTO, Mapper, Repository, Service et Controller, en appliquant la même architecture modulaire que le module Product.



Customer.java:

```java
package com.ecommerce.monolith.customer.model;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Table(name = "customers")
@Getter @Setter
@NoArgsConstructor @AllArgsConstructor
@Builder
public class Customer {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(nullable = false, unique = true)
    private String email;

    private String phone;
}
```

CustomerDTO.java:

```java
package com.ecommerce.monolith.customer.dto;

import lombok.Data;

@Data  no usages
public class CustomerDTO {
    private Long id;
    private String name;
    private String email;
    private String phone;
}
```

CreateCustomerRequest.java:

```java
package com.ecommerce.monolith.customer.dto;

import jakarta.validation.constraints.*;
import lombok.Data;

@Data  no usages
public class CreateCustomerRequest {
    @NotBlank(message = "Name is required")
    private String name;

    @NotBlank(message = "Email is required")
    @Email(message = "Email is invalid")
    private String email;

    private String phone;
}
```

CustomerMapper.java:

```java
package com.ecommerce.monolith.customer.mapper;

import com.ecommerce.monolith.customer.dto.CreateCustomerRequest;
import com.ecommerce.monolith.customer.dto.CustomerDTO;
import com.ecommerce.monolith.customer.model.Customer;
import org.mapstruct.*;
import java.util.List;

@Mapper(componentModel = "spring")  no usages  1 implementation
public interface CustomerMapper {
    CustomerDTO toDTO(Customer customer);  no usages  1 implementation
    List<CustomerDTO> toDTOList(List<Customer> customers);  no usages  1 implementation
    Customer toEntity(CreateCustomerRequest request);  no usages  1 implementation
}
```

CustomerRepository.java:

```java
package com.ecommerce.monolith.customer.repository;

import com.ecommerce.monolith.customer.model.Customer;
import org.springframework.data.jpa.repository.JpaRepository;

public interface CustomerRepository extends JpaRepository<Customer, Long> {
    boolean existsByEmail(String email);
}
```

CustomerService.java:

```java
package com.ecommerce.monolith.customer.service;

import com.ecommerce.monolith.customer.dto.CreateCustomerRequest;
import com.ecommerce.monolith.customer.dto.CustomerDTO;
import java.util.List;

public interface CustomerService {
    List<CustomerDTO> getAllCustomers();
    CustomerDTO getCustomerById(Long id);
    CustomerDTO createCustomer(CreateCustomerRequest request);
    boolean customerExists(Long id); // vérifie l'existence
    void deleteCustomer(Long id);
}
```

CustomerServiceImpl.java:

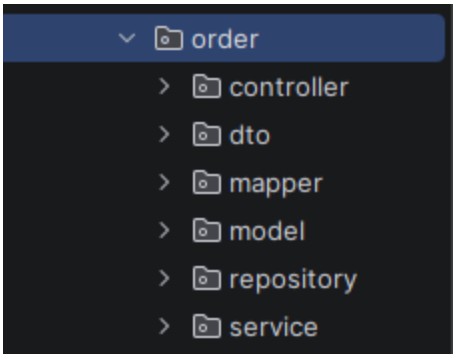
```java
package com.ecommerce.monolith.customer.service;

import com.ecommerce.monolith.customer.dto.CreateCustomerRequest;
import com.ecommerce.monolith.customer.dto.CustomerDTO;
import com.ecommerce.monolith.customer.mapper.CustomerMapper;
import com.ecommerce.monolith.customer.model.Customer;
import com.ecommerce.monolith.customer.repository.CustomerRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
@RequiredArgsConstructor
public class CustomerServiceImpl implements CustomerService {
    private final CustomerRepository repository;
    private final CustomerMapper mapper;

    @Override  no usages
    public List<CustomerDTO> getAllCustomers() {
        return mapper.toDTOList(repository.findAll());
    }

    @Override  no usages
    public CustomerDTO getCustomerById(Long id) {
        Customer customer = repository.findById(id)
                .orElseThrow(() -> new RuntimeException("Customer not found with id: " + id));
        return mapper.toDTO(customer);
    }

    @Override  no usages
    public CustomerDTO createCustomer(CreateCustomerRequest request) {
        Customer customer = mapper.toEntity(request);
        return mapper.toDTO(repository.save(customer));
    }

    @Override  no usages
    public boolean customerExists(Long id) {
        return repository.existsById(id);
    }

    @Override  no usages
    public void deleteCustomer(Long id) {
        if (!repository.existsById(id))
            throw new RuntimeException("Customer not found with id: " + id);
        repository.deleteById(id);
    }
}
```

CustomerController.java:

```java
package com.ecommerce.monolith.customer.controller;

import com.ecommerce.monolith.customer.dto.CreateCustomerRequest;
import com.ecommerce.monolith.customer.dto.CustomerDTO;
import com.ecommerce.monolith.customer.service.CustomerService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/customers")
@RequiredArgsConstructor
public class CustomerController {
    private final CustomerService customerService;

    @GetMapping
    public ResponseEntity<List<CustomerDTO>> getAll() {
        return ResponseEntity.ok(customerService.getAllCustomers());
    }

@GetMapping("/{id}")
public ResponseEntity<CustomerDTO> getById(@PathVariable Long id) {
    return ResponseEntity.ok(customerService.getCustomerById(id));
}

@PostMapping
public ResponseEntity<CustomerDTO> create(@Valid @RequestBody CreateCustomerRequest request) {
    return ResponseEntity.status(HttpStatus.CREATED).body(customerService.createCustomer(request));
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> delete(@PathVariable Long id) {
    customerService.deleteCustomer(id);
    return ResponseEntity.noContent().build();
}
```

Partie 2 — Module Order:

J'ai créé le module Order qui dépend des services publics de Product et Customer sans accéder directement à leurs repositories, respectant ainsi le principe de séparation des modules.



Order.java:

```java
package com.ecommerce.monolith.order.model;

import com.ecommerce.monolith.customer.model.Customer;
import com.ecommerce.monolith.product.model.Product;
import jakarta.persistence.*;
import java.math.BigDecimal;
import java.time.LocalDateTime;

@Entity
@Table(name = "orders")
public class Order {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne  2 usages
    @JoinColumn(name = "customer_id", nullable = false)
    private Customer customer;

    @ManyToOne  2 usages
    @JoinColumn(name = "product_id", nullable = false)
    private Product product;

    @Column(nullable = false)  2 usages
    private Integer quantity;
```

```java
    @Column(nullable = false)  2 usages
    private BigDecimal totalPrice;

    @Column(nullable = false)  2 usages
    private LocalDateTime orderDate = LocalDateTime.now();

    public Order() {}

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public Customer getCustomer() { return customer; }
    public void setCustomer(Customer customer) { this.customer = customer; }
    public Product getProduct() { return product; }
    public void setProduct(Product product) { this.product = product; }
    public Integer getQuantity() { return quantity; }  1 usage
    public void setQuantity(Integer quantity) { this.quantity = quantity; }  1 usage
    public BigDecimal getTotalPrice() { return totalPrice; }  1 usage
    public void setTotalPrice(BigDecimal totalPrice) { this.totalPrice = totalPrice; }  1 usage
    public LocalDateTime getOrderDate() { return orderDate; }  1 usage
    public void setOrderDate(LocalDateTime orderDate) { this.orderDate = orderDate; }  no usages

    public static OrderBuilder builder() { return new OrderBuilder(); }  1 usage

    public static class OrderBuilder {  6 usages
        private Customer customer;  2 usages
        private Product product;  2 usages
        private Integer quantity;  2 usages
        private BigDecimal totalPrice;  2 usages

        public OrderBuilder customer(Customer customer) { this.customer = customer; return this; }  no usages
        public OrderBuilder product(Product product) { this.product = product; return this; }  no usages
        public OrderBuilder quantity(Integer quantity) { this.quantity = quantity; return this; }  no usages
        public OrderBuilder totalPrice(BigDecimal totalPrice) { this.totalPrice = totalPrice; return this; }

        public Order build() {  1 usage
            Order order = new Order();
            order.setCustomer(this.customer);
            order.setProduct(this.product);
            order.setQuantity(this.quantity);
            order.setTotalPrice(this.totalPrice);
            return order;
        }
    }
}
```

OrderDTO.java:

```java
package com.ecommerce.monolith.order.dto;

import lombok.Data;
import java.math.BigDecimal;
import java.time.LocalDateTime;

@Data  no usages
public class OrderDTO {
    private Long id;
    private Long customerId;
    private Long productId;
    private Integer quantity;
    private BigDecimal totalPrice;
    private LocalDateTime orderDate;
}
```

CreateOrderRequest.java:

```java
package com.ecommerce.monolith.order.dto;

import jakarta.validation.constraints.*;
import lombok.Data;

@Data  no usages
public class CreateOrderRequest {
    @NotNull(message = "Customer ID is required")
    private Long customerId;

    @NotNull(message = "Product ID is required")
    private Long productId;

    @NotNull @Positive(message = "Quantity must be greater than 0")
    private Integer quantity;
}
```

OrderMapper.java:

```java
package com.ecommerce.monolith.order.mapper;

import com.ecommerce.monolith.order.dto.OrderDTO;
import com.ecommerce.monolith.order.model.Order;
import org.mapstruct.*;
import java.util.List;

@Mapper(componentModel = "spring")  no usages  1 implementation
public interface OrderMapper {
    @Mapping(source = "customer.id", target = "customerId")  no usages  1 implementation
    @Mapping(source = "product.id", target = "productId")
    OrderDTO toDTO(Order order);
    List<OrderDTO> toDTOList(List<Order> orders);  no usages  1 implementation
}
```

OrderRepository.java:

```java
package com.ecommerce.monolith.order.repository;

import com.ecommerce.monolith.order.model.Order;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface OrderRepository extends JpaRepository<Order, Long> {  no usages
    List<Order> findByCustomerId(Long customerId); // historique commandes  no usages
}
```

OrderService.java:

```java
package com.ecommerce.monolith.order.service;

import com.ecommerce.monolith.order.dto.CreateOrderRequest;
import com.ecommerce.monolith.order.dto.OrderDTO;
import java.util.List;

public interface OrderService {  no usages  1 implementation
    OrderDTO createOrder(CreateOrderRequest request);  no usages  1 implementation
    List<OrderDTO> getOrdersByCustomer(Long customerId); // historique  no usages  1 implementation
    List<OrderDTO> getAllOrders();  no usages  1 implementation
}
```

OrderServiceImpl.java:

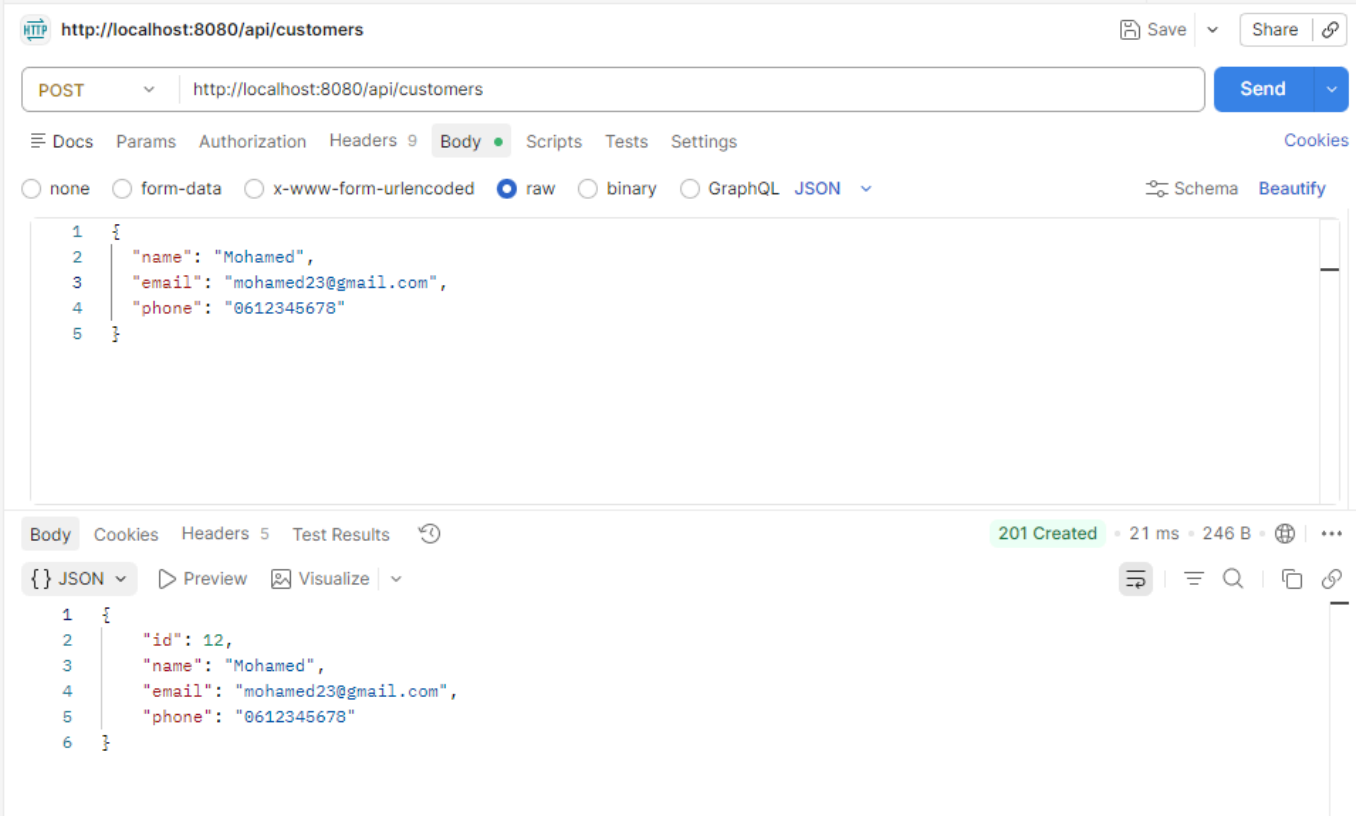
```java
package com.ecommerce.monolith.order.service;

import com.ecommerce.monolith.customer.model.Customer;
import com.ecommerce.monolith.customer.repository.CustomerRepository;
import com.ecommerce.monolith.order.dto.CreateOrderRequest;
import com.ecommerce.monolith.order.dto.OrderDTO;
import com.ecommerce.monolith.order.mapper.OrderMapper;
import com.ecommerce.monolith.order.model.Order;
import com.ecommerce.monolith.order.repository.OrderRepository;
import com.ecommerce.monolith.product.model.Product;
import com.ecommerce.monolith.product.repository.ProductRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
@RequiredArgsConstructor
public class OrderServiceImpl implements OrderService {
    private final OrderRepository orderRepository;
    private final CustomerRepository customerRepository;
    private final ProductRepository productRepository;
    private final OrderMapper mapper;

    @Override  1 usage
    public OrderDTO createOrder(CreateOrderRequest request) {
        Customer customer = customerRepository.findById(request.getCustomerId())
                .orElseThrow(() -> new RuntimeException("Customer not found"));
        Product product = productRepository.findById(request.getProductId())
                .orElseThrow(() -> new RuntimeException("Product not found"));

        Order order = Order.builder()
                .customer(customer)
                .product(product)
                .quantity(request.getQuantity())
                .totalPrice(product.getPrice().multiply(
                        java.math.BigDecimal.valueOf(request.getQuantity())))
                .build();

        return mapper.toDTO(orderRepository.save(order));
    }

    @Override  1 usage
    public List<OrderDTO> getOrdersByCustomer(Long customerId) {
        if (!customerRepository.existsById(customerId))
            throw new RuntimeException("Customer not found");
        return mapper.toDTOList(orderRepository.findByCustomerId(customerId));
    }

    @Override  1 usage
    public List<OrderDTO> getAllOrders() {
        return mapper.toDTOList(orderRepository.findAll());
    }
}
```

OrderController.java:

```java
package com.ecommerce.monolith.order.controller;
import com.ecommerce.monolith.order.dto.CreateOrderRequest;
import com.ecommerce.monolith.order.dto.OrderDTO;
import com.ecommerce.monolith.order.service.OrderService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;
@RestController
@RequestMapping("/api/orders")
@RequiredArgsConstructor
public class OrderController {
    private final OrderService orderService;
    @PostMapping
    public ResponseEntity<OrderDTO> create(@Valid @RequestBody CreateOrderRequest request) {
        return ResponseEntity.status(HttpStatus.CREATED).body(orderService.createOrder(request));
    }
    @GetMapping
    public ResponseEntity<List<OrderDTO>> getAll() {
        return ResponseEntity.ok(orderService.getAllOrders());
    }
    @GetMapping("/customer/{customerId}")
    public ResponseEntity<List<OrderDTO>> getByCustomer(@PathVariable Long customerId) {
        return ResponseEntity.ok(orderService.getOrdersByCustomer(customerId));
```

Test Postman — Customer:

J'ai testé la création d'un customer via POST `/api/customers` qui retourne 201 Created, confirmant que le module fonctionne correctement.



Test Postman — Order:

J'ai testé la création d'une commande via POST `/api/orders` en associant un customer et un produit existants, le total est calculé automatiquement en multipliant le prix par la quantité.

http://localhost:8080/api/products

Save Share

POST http://localhost:8080/api/products Send

Docs Params Authorization Headers 9 Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "name": "Laptop",
3   "description": "Un bon laptop",
4   "price": 999.99,
5   "stock": 10
6 }
```

Body Cookies Headers 5 Test Results 201 Created 59 ms 250 B

{} JSON Preview Visualize

```
1 {
2     "id": 15,
3     "name": "Laptop",
4     "description": "Un bon laptop",
5     "price": 999.99,
6     "stock": 10
7 }
```

Test Postman — Historique commandes:

J'ai testé GET `/api/orders/customer/{id}` qui retourne bien l'historique des commandes d'un client spécifique, validant ainsi la relation entre les modules.

http://localhost:8080/api/orders

Save Share

POST http://localhost:8080/api/orders Send

Docs Params Authorization Headers 9 Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "customerId": 12,
3   "productId": 15,
4   "quantity": 2
5 }
```

Body Cookies Headers 5 Test Results 201 Created 55 ms 284 B

{} JSON Preview Visualize

```
1 {
2     "id": 1,
3     "customerId": 12,
4     "productId": 15,
5     "quantity": 2,
6     "totalPrice": 1999.98,
7     "orderDate": "2026-04-02T04:05:08.0405845"
8 }
```

Résumé:
Cet exercice m'a permis d'implémenter une architecture modulaire complète avec trois modules interdépendants, où Order utilise les services publics de Product et Customer sans couplage direct entre les repositories